TinyShell: Shortcuts, Scaling Limits, and Prunable Structure in a 10M-Parameter Natural-Language-to-Shell Model

Rupesh Kumar
Code, data pipeline, checkpoints, and evaluation logs:
https://github.com/Rupesh-rkgit/tinyshell

October 2026

Abstract

We present TinyShell, an end-to-end study of natural-language-to-shell models trained from scratch on a CPU-only budget, culminating in a 9.6M-parameter decoder trained on 100M tokens derived from tldr-pages. Because we control the data generator, we hold out slot values, request phrasings, and entire command pages independently, pair every neural result with a BM25 retrieval baseline, and report page-clustered confidence intervals. Five findings emerge. (1) Naive sequence packing teaches a cross-example copying shortcut that validation loss hides; evaluating the frozen checkpoint under a block-diagonal mask lowers loss by 0.56 nats, exposing it without retraining. (2) At 2.1M parameters, command-family selection is label-space limited ($0.36 \rightarrow 0.87$ when 3,651 first tokens are restricted to 111) while argument composition is capacity limited. (3) Scaling 4.6× in parameters and 10.9× in tokens lifts seen-command exact match to 0.525–0.559 but leaves unseen-page exact match at 0.005 and *worsens* unseen-page loss ($3.80 \rightarrow 4.15$). (4) Against retrieval, the model's unseen-page token-F1 advantage is significant on only one of three unseen splits, while on seen pages the two systems' errors are complementary (oracle routing: 0.914 vs. 0.843 exact). (5) Opening the trained network with unit masks, we find that attention in layers 0–2 reads the request and early heads copy its entities, that MLPs at every depth hold command knowledge, and that unseen-page predictions stall mid-network. Post-training pruning exposes a small-model regime unlike the LLM one: *no* scoring rule beats random for MLP neurons (10% removal already costs 5–9 exact points), whereas Fisher-scored heads with global allocation can be cut by 25% for 7 points; and seen-page knowledge is not localized—removing the units most specific to seen pages is no more selective than random. Short masked distillation recovers only part of the loss, and after it the scoring rule no longer matters; but a subnetwork scored and distilled on ten DevOps command families yields a 2.95M-parameter specialist that keeps 65% of the generalist's exact match on those families (0.300 vs. 0.465) and selects the family more accurately (0.91 vs. 0.79). We release the full pipeline, probing and pruning harness, checkpoints, and per-example logs as a reproducible testbed for small-model research.

1 Introduction

Mapping an English request to a single shell command is a bounded, checkable, and practically useful instance of semantic parsing: outputs are short (median 26 characters), correctness is measurable without human judges, and the resource a model must possess—a vocabulary of tools and flags—is small enough to study exhaustively. Prior work established the task with curated corpora and neural semantic parsers [1, 2]; modern code LLMs solve it broadly but opaquely at billions of parameters [3, 4]. What is missing is a *small, fully instrumented* testbed in which every design decision—corpus construction, tokenization, packing, objective, split design, baselines, and the internal structure of the trained network—is visible, ablatable, and honest

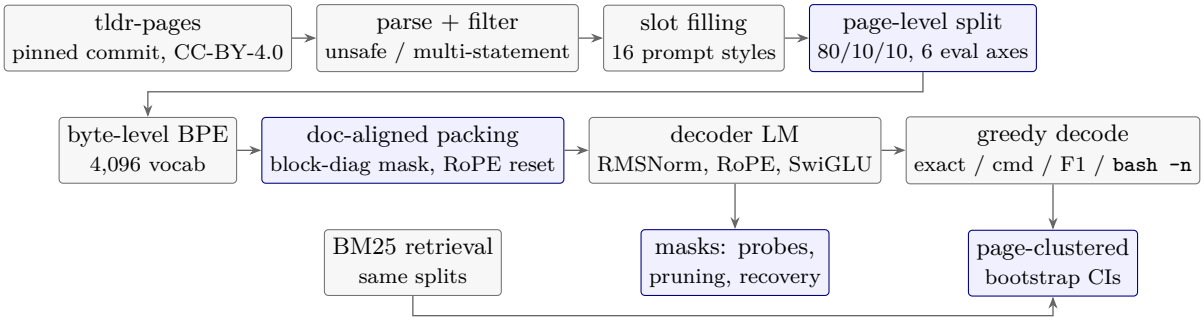


Figure 1: The TinyShell pipeline. Shaded stages are the methodological contributions: page-level holdout along orthogonal axes, document-aligned packing, clustered significance testing against retrieval, and post-hoc structural analysis of the trained network via unit masks. Every stage is released code; every number in this paper is reproducible from the pinned corpus commit and per-run metadata.

about failure. TinyShell (Figure 1) is such a testbed. Because we control the data generator, we can hold out slot values, request phrasings, and entire command pages independently, and because the model is small (2.1M–9.6M parameters, trained on CPU), we can afford exhaustive interventions on it.

We organize the study around five research questions:

RQ1 *Training signal.* Can a standard efficiency recipe silently change the conditional distribution a model learns, and can this be diagnosed without retraining? (Section 6)

RQ2 *Failure decomposition.* Is the residual failure of a tiny model limited by the size of the label space or by capacity? (Section 7)

RQ3 *Scaling.* What does 4.6× parameters and 10.9× tokens buy, and along which generalization axis? (Section 8)

RQ4 *Retrieval control.* Does the generative model outperform nearest- neighbour retrieval anywhere, under clustered significance testing? (Section 9)

RQ5 *Internal structure.* Where in the network is the command family decided and the arguments composed; how much of it can be removed after training; and are page-specific memories localized enough to be pruned selectively? (Section 10)

Our contributions:

- **A shortcut diagnosis with an eval-time causal test** (**RQ1**): naive sequence packing teaches cross-example copying that validation loss conceals; a frozen-checkpoint mask ablation (3.866 → 3.306 nats) and a behavioural prefix probe identify it without retraining.
- **A label-space/capacity decomposition** (**RQ2**): restricting 3,651 first tokens to 111 raises family selection from 0.36 to 0.87 at fixed architecture, while argument composition stays the bottleneck.
- **A scaling result along orthogonal axes** (**RQ3**): seen-command exact match rises from ≤ 0.002 to 0.525–0.559, unseen-page exact match stays at 0.005, and unseen-page loss *regresses* (3.80 → 4.15).
- **A significance-tested retrieval accounting** (**RQ4**): with page-clustered paired bootstrap intervals, the generative model's token-F1 advantage over BM25 on unseen pages is significant on one of three unseen splits only; on seen splits, the two systems' errors are complementary (an oracle router would reach 0.914 exact vs. 0.843 for BM25).
- **A mechanistic and pruning study in the small-relative-to-data regime** (**RQ5**): early attention reads and copies, MLPs at every depth store command knowledge; no MLP scoring rule beats random (10% removal costs 5–9 exact points) while Fisher-scored heads with global allocation can be cut by 25% for 7 points; page knowledge is not localized; and a family-scored, distilled 2.95M subnetwork keeps 65% of the dense model's exact match on its families.
- **A released platform**: data and model cards, pinned corpus commit, per-run metadata,

checkpoints, raw per-example predictions, the probing/pruning harness, and a 16-chapter learning book.

2 Related Work

NL-to-shell. Lin et al. [1] built Tellina, an RNN semantic parser with slot filling for bash one-liners, and Lin et al. [2] released NL2Bash (∼9,305 expert-described commands). TinyShell differs in source (curated tldr-pages), scale (from-scratch 2–10M parameters on CPU), and focus (generalization audit, baselines, and mechanism rather than leaderboard performance).

Small and instruction-tuned code models. Codex [3], Code Llama [4], the Llama family [5, 6], and instruction tuning [7] solve shell questions broadly; we operate three orders of magnitude smaller to keep every mechanism measurable, in the spirit of TinyStories [8], TinyLlama [9], and nanoGPT [10].

Sequence packing. Packing variable-length samples into fixed rows is standard [11, 12]; block-diagonal masking with position reset restores equivalence to unpacked training [11] and is used in conditioned generation [13]. Our contribution is diagnostic: the contamination left by naive packing is a learned conditional that silently changes the task, measurable by evaluating a frozen checkpoint under both masks.

Scaling and data mixture. Scaling analyses [14, 15] concern tokens and parameters for a fixed distribution; we report a regime in which more tokens (more phrasings of known pages) improve interpolation and regress extrapolation. Retrieval augmentation [16, 17] and execution environments [18] frame our proposed remedies.

Pruning. Saliency-based pruning dates to Optimal Brain Damage [19]; magnitude pruning with retraining [20] and the lottery-ticket hypothesis [21] established that trained networks contain sparse sub-networks. First-order Taylor saliencies [22] and mask-gradient head importance [23] showed that many attention heads can be removed, and that the important ones are specialized [24]; Kwon et al. [25] use the Fisher information of unit masks for retraining-free structured pruning. At LLM scale, SparseGPT [26] and Wanda [27] prune weights one-shot, LLM-Pruner [28], SliceGPT [29], Sheared LLaMA [30], and Minitron [31] remove structure and recover with continued training or distillation [32], and whole deeper layers are often redundant [33, 34]. Hooker et al. [35] showed that compression disproportionately harms rare examples. This literature studies heavily over-parameterized models; we ask what changes when a model is *small relative to its data*.

Interpretability and memorization. The logit lens [36] and tuned lens [37] decode intermediate residual streams; circuit analyses [38, 39] identify copying and induction mechanisms [40]; feed-forward layers act as key–value memories [41] that store factual associations [42, 43]. Memorization is necessary for long-tail learning [44], and whether it is localized in a few units is contested [45, 46]. Our seen-vs-unseen differential pruning is a direct test of localization in a setting where "memorized" (seen pages) and "novel" (unseen pages) are defined by construction.

3 Task and Data

3.1 Corpus

The sole knowledge source is tldr-pages [47], pinned at commit `95062b79` (`pages/common` + `pages/linux`; CC-BY-4.0): 6,803 pages / 30,308 raw examples yield 6,621 usable pages / 28,659 usable base examples after filtering. Each example is a one-line description plus a command template with `{{placeholder}}` slots. Filters reject multi-statement, substitution-heavy, history-expanded, interactive, and oversized commands; the run-3 revision additionally rejects cron-shaped first tokens, leading `!`, and process substitution (−139 examples). Slots are filled

Split	Pages	Prompt style	Measures
`val`	seen	8 (held out)	checkpoint selection
`val_unseen`	unseen	seen	command generalization (loss)
`test_seen`	seen	9 (held out) + new slots	slot filling + style robustness
`test_paraphrase`	seen	10–11 (unseen)	phrasing robustness
`test_unseen`	unseen	seen	command generalization
`test_unseen_paraphrase`	unseen	10–11	both axes held out

Table 1: Evaluation splits. Holding out slots, styles, and pages independently is what lets the scaling analysis of Section 8 attribute gains and failures to a specific axis.

	2.1M (runs 1–2, core)	9.6M (run 3)
d_{model} / layers / heads	128 / 6 / 4	256 / 10 / 8
Head dim / SwiGLU width	32 / 512	32 / 768
Context / vocab	256 / 4,096	256 / 4,096
Dropout	0.05 (core 0.1)	0.05
Total parameters	2,098,816	**9,573,632**
Non-embedding (MLP / attention)	1,574,528	8,525,056 (5.90M / 2.62M)

Table 2: Architectures. Head dimension is fixed at 32 across scales. In the 9.6M model, MLPs hold 62% of all parameters, which makes MLP neurons the natural unit for structured pruning (Section 10).

from hand-written pools (file, path, user, host, package, port, pattern, version, and fixed synthetic credentials) with per-variant fresh values and tldr alternative-syntax resolution.

3.2 Requests, styles, and splits

Each base example is rendered into requests by 16 templates ("styles"): 12 train styles (0–7, 12–15) and styles 8–11 *held out* for evaluation. Slot mentions in the description are substituted inline; remaining slots are declared in an appended context sentence ("Assume the file is `/tmp/notes.txt`."). Splits are by *page* (deterministic hash of the page title, 80/10/10), so all examples of a command fall in exactly one split. Six evaluation splits of 1,500 requests each orthogonalize three axes (Table 1).

3.3 Versions

Dataset v1 (runs 1–2, core): 6 variants per base example, 135,860 training requests, 9.16M packed tokens per epoch. Dataset XL (run 3): 150 variants per example with 4 additional train styles, 1,303,083 requests, 99,992,950 packed training tokens in 468,212 document-aligned chunks (17,411,259 supervised tokens; longest example 229 tokens). Slot-free examples saturate at 12 distinct phrasings; slot-bearing examples scale near-linearly. The same page split is preserved across versions. Risk labels (129,177 safe / 14,822 caution / 861 destructive) are regex heuristics used for analysis and CLI warnings, never as training targets; `bash -n` validity of a 2,000-command sample is 99.85%.

4 Model, Objective, and Training

TinyShell is a pre-norm decoder-only transformer [5, 48]: RMSNorm [49], RoPE [50], fused scaled-dot-product attention [12], SwiGLU MLPs [51], and tied embeddings [52] (Table 2). A byte-level BPE [53] with a 4,096 vocabulary and a fixed system/user/assistant template is shared by all runs; prompt tokens are masked with -100, so the loss covers only the command and EOS.

	Run 1 (naive)	Run 2 (doc-aligned)	Δ
Best val loss	3.9749	**2.9377**	−1.04
Val loss, unseen pages	4.6025	**3.7993**	−0.80
Exact match (best split)	0.002	0.002	≈ 0
Command match (best split)	0.089	0.089	≈ 0
Token F1 (best split)	0.128	**0.187**	+0.06
`sudo`-prefixed predictions (`test_seen`)	54%	42%	−12 pts

Table 3: Run 1 vs. run 2. Removing the shortcut lowers loss by more than a nat and improves argument copying everywhere, but exact match stays ≈ 0.

Objective. Token cross-entropy on supervised positions. An optional *first-token weight* w multiplies the loss on the first command token (the command-family decision); w=5 in the core diagnostic, w=1 elsewhere. We report this decomposition throughout: *first-token* loss measures family selection, *argument* loss measures composition.

Training. AdamW [54] (decay 0.1 on matrices only, β=(0.9, 0.95)), lr 3e−4 cosine-annealed to 3e−5 after 200–300 warmup steps, gradient clipping at 1.0, fp32 on 4 vCPU ARM64 (Neoverse-N1). Examples are packed complete-only into 256-token rows (∼14% padding). Best checkpoints are selected on `val`. Seed 42 throughout; per-run config, environment (Python 3.12, torch 2.14), git commit, and tokenizer snapshots are stored with the logs.

5 Evaluation Protocol and Statistics

Greedy decoding, 64 new tokens maximum. *Exact*: normalized prediction equals the reference. *Command*: first executable token matches (wrappers such as `sudo` skipped). *Token F1*: multiset F1 over arguments. *Valid bash*: `bash -n` parses (syntax only; nothing is executed). The BM25 baseline [55] returns the command of the most similar training request.

Uncertainty. Proportions carry Wilson 95% intervals [56]; at n=1,500 the half-width is at most 2.5 points. Model-vs-BM25 differences are paired (same requests) and are reported with 95% percentile bootstrap intervals [57] with 10^4 resamples. Because all examples of a page share a command, examples are not independent; we therefore resample *pages* (595–1,318 clusters per split) rather than examples. All runs are single-seed; we flag this as a threat to validity in Section 11.

6 The Packing Shortcut and Its Repair (RQ1)

Run 1 (naive packing). Concatenating all examples into one stream and slicing it into 256-token rows lets position t of example N attend to the *command of example* N−1. The model reached a plausible loss (best val 3.9749) but 0.000 exact match, with 72% of predictions beginning with `sudo` (references: 6%), e.g. *"list all files including hidden ones"* → `sudo aa-cli --list`.

Diagnosis without retraining. Two probes on the frozen checkpoint: (i) a *prefix probe*—prepending an unrelated prior example flips the prediction to copy it; (ii) a *mask ablation*—evaluating the same packed validation set with a block-diagonal document mask lowers validation loss from 3.866 to 3.306 (−0.56 nats). If preceding examples were useful context, masking them would *hurt*; the improvement shows a harmful learned dependence. Both probes are possible because document ids are a runtime input.

Run 2 (document-aligned packing). Retraining with complete-example packing, per-token document ids, within-example positions, and the block-diagonal causal mask [11]—same architecture, data, and optimizer—lowers loss by more than a nat (Table 3).

Split (core)	N	Exact	Command	Token F1	Valid bash
`val`	187	0.048	**0.866**	0.227	0.952
`test_seen`	179	0.028	**0.737**	0.155	0.983
`test_paraphrase`	211	0.057	**0.924**	0.260	0.981

Table 4: Core-subset diagnostic (2.1M). Command match rises from 0.36 (the broad run-2 model scored on the same 187 `val` examples) to 0.74–0.92; `sudo` over-prediction drops from 42% to 6.7%; exact match stays at 0.028–0.057 because 71% of `test_seen` errors are right-family-wrong-arguments. Note the small n: Wilson intervals on exact match are about ± 3 points.

	Run 1	Run 2	Run 3
Best val loss (seen pages)	3.9749	2.9377	**0.6565**
Val loss, unseen pages	4.6025	**3.7993**	4.1451
Seen/unseen gap (nats)	0.63	0.87	**3.49**

Table 5: Validation loss. Run 3 *regresses* on unseen pages while seen loss collapses; the seen/unseen gap widens from 0.87 to 3.49 nats.

Finding (RQ1) Naive packing changed the learned conditional from $P(\text{cmd} \mid \text{request})$ to $P(\text{cmd} \mid \text{request}, \text{previous example})$. Validation loss did not reveal this; generation probes and a frozen-checkpoint mask ablation did. The repair is standard, but the diagnostic generalizes: *evaluate a frozen model under the inference-time attention pattern before trusting its training loss.*

7 Label Space vs. Capacity (RQ2)

We build a focused subset—pages of ∼100 curated common tools: 655 pages, 17,317 requests, **111 distinct first command tokens** (vs. 3,651 broad)—and train the identical 2.1M model for 20 epochs with dropout 0.1 and first-token weight 5.0 (2.2 h, Table 4).

Finding (RQ2) At 2.1M parameters, family selection is *label-space limited* and argument composition is *capacity limited*. The comparison is not a controlled ablation (the core training set is a slice of the broad corpus), but it redirected the run-3 budget toward capacity and data rather than architecture.

8 Scaling: 9.6M Parameters, 100M Tokens (RQ3)

Run 3 scales the same design 4.6× in parameters over the XL corpus (10.9× tokens; stricter filters; four extra train styles; held-out eval styles and page split unchanged), trained one pass in 27.7 h at ∼1,190 tokens/s: 3,657 steps, best val loss **0.6565** (perplexity 1.93) at step 3,500.

Error census. On `test_seen`: 788 exact (52.5%), 250 right-command-wrong-arguments (16.7%), 4 invalid bash, 0 empty. The `sudo` prior persists at 14.7% of predictions vs. 5.5% of references (run 2: 42%), concentrated on obscure admin pages. Seen-split residue is dominated by sibling-tool confusions (`phpcs`↔`phpcbf`, `7zr`↔`7za`) and invented flags on correct families.

Finding (RQ3) Scaling along the *phrasing* axis (150 variants per page) bought interpolation—slot filling and paraphrase robustness—but not extrapolation: unseen-page exact match stayed at 0.005 and unseen-page loss got worse. More tokens of known pages sharpen the fit to those pages; they do not supply the flag vocabularies of tools the model has never seen.

Split	Exact [95%, CI]	Command	Token F1	Valid bash	p50 lat.
`val`	0.460 [.434, .485]	0.618	0.604	0.997	290 ms
`val_unseen`	0.011 [.007, .017]	0.138	0.315	0.995	227 ms
`test_seen`	**0.525** [.500, .550]	0.692	0.644	0.997	245 ms
`test_paraphrase`	**0.559** [.534, .584]	**0.722**	**0.681**	0.997	242 ms
`test_unseen`	0.005 [.002, .010]	0.111	0.289	0.989	229 ms
`test_unseen_para.`	0.005 [.003, .011]	0.098	0.292	0.991	242 ms

Table 6: Run 3 (`best` checkpoint, greedy, n=1,500 per split; Wilson intervals). Seen-command composition now works and is robust to unseen phrasings; unseen-page exact match is unchanged from run 2.

	Exact		Token F1		Exact discordant	
Split	BM25	Δ model−BM25	BM25	Δ model−BM25	model only	BM25 only
`val`	0.851	−0.392 [−.423, −.361]	0.947	−0.343 [−.367, −.320]	85	672
`val_unseen`	0.009	+0.001 [−.003, +.005]	0.286	**+0.029** [+.012, +.046]	6	4
`test_seen`	0.843	−0.317 [−.350, −.286]	0.918	−0.274 [−.300, −.250]	106	582
`test_paraphrase`	0.877	−0.317 [−.348, −.287]	0.964	−0.283 [−.306, −.260]	82	558
`test_unseen`	0.007	−0.003 [−.007, +.001]	0.277	+0.011 [−.005, +.027]	3	7
`test_unseen_para.`	0.008	−0.003 [−.008, +.003]	0.291	+0.001 [−.016, +.018]	4	8

Table 7: Run 3 vs. BM25 on identical requests. Δ columns give paired differences with 95% page-clustered bootstrap intervals (10^4 resamples). Only one unseen-split advantage excludes zero. The last two columns count requests that exactly one system gets right.

9 Generative Model vs. Retrieval (RQ4)

BM25 is near-oracle on seen splits by construction: evaluation requests are generated paraphrases of training requests, so the nearest training request usually carries the reference command. On unseen pages it is near-useless ($\leq 0.8\%$ exact) because the correct command is not in its index. Table 7 shows two things that point estimates alone hide.

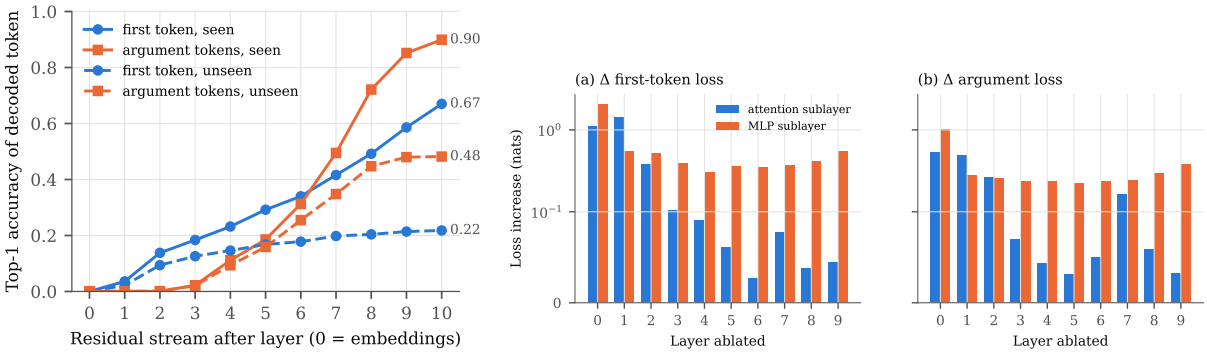
First, the generative model's token-F1 advantage on unseen pages is *statistically supported only on* ***`val_unseen`*** (+0.029, [+0.012, +0.046]); on both test-unseen splits the interval includes zero. An earlier version of this analysis read the three positive point estimates as consistent evidence of composition; with clustered intervals, the evidence is suggestive, not established.

Second, the two systems' errors on seen pages are *complementary*: on `test_seen` the model is exactly right on 106 requests that BM25 gets wrong. An oracle that picks the right system per request would reach $(1265 + 106)/1500 = 0.914$ exact, versus 0.843 for BM25 and 0.525 for the model. This bounds what a confidence-based router (Section 12) could gain.

Finding (RQ4) Retrieval dominates seen pages, neither system works on unseen pages, and the generative model's edge on unseen-page token F1 is real on one of three splits. Clustered resampling matters: page-level dependence widens intervals and changes which claims survive.

10 Inside the Model: Probes and Post-Training Pruning (RQ5)

We now open the 9.6M run-3 checkpoint. All interventions use *multiplicative unit masks* installed as forward hooks: an MLP neuron mask $m^{\text{mlp}}_{\ell,i}$ scales SwiGLU hidden unit i of layer ℓ before the down-projection, and a head mask $m^{\text{att}}_{\ell,h}$ scales head h's output before the output projection. Setting a mask to zero is exactly structured removal: the corresponding row of the gate/up matrices and column of the down matrix (or the head's q, k, v, o slices) no longer affect the output, and we verify that physically slicing the matrices reproduces the masked model to $< 10^{-5}$ in the logits.



(a) Logit lens. **(b)** Zero-ablating one sublayer (seen pages).

Figure 2: (a) Top-1 accuracy of decoding the residual stream after each layer with the final norm and unembedding. Argument tokens become decodable late (layers 7–10); on unseen pages the family token plateaus at ≈ 0.2 from layer 6 on. **(b)** Loss increase from removing an entire attention or MLP sublayer (symlog scale). Attention matters in layers 0–2 and little thereafter; every MLP matters.

10.1 Protocol

Unit scores. We compare four scoring rules for which units to remove: *random*; *magnitude* (product of the unit's incoming and outgoing weight norms); *activation×weight* (mean |unit output| times outgoing norm, a structured analogue of Wanda [27]); and the *empirical Fisher* of the mask, $F_u = \mathbb{E}_x[(\partial \mathcal{L}(x)/\partial m_u)^2]$ at $m{=}1$ [23, 25], whose half approximates the loss increase from removing u. Magnitude and activation scores are not comparable across layers, so they are used with a *uniform* per-layer budget; Fisher scores are in loss units and are also used with a single *global* ranking that lets the score decide each layer's share.

Data hygiene. Scores are computed on 512 *training* requests (seen pages) and, for the seen/unseen comparison, on 512 `val_unseen` requests; all reported metrics use disjoint sets: teacher-forced loss on the first 500 `val` (seen) and `test_unseen` requests, and greedy generation on the first 200 `test_seen` and `test_unseen` requests. Batched KV-cached decoding is verified against the reference decoder (100% agreement). Loss is split into the *first command token* (family selection) and the remaining *argument* tokens, as in Section 7.

10.2 Where the decisions are made

Read early, look up everywhere. Removing the attention sublayer of layers 0, 1, or 2 raises first-token loss by +1.10, +1.44, and +0.38 nats; for layers 3–9 the increase is at most +0.10 (Figure 2b). MLP sublayers are uniformly important (between +0.31 and +0.56 nats each), with layer 0's MLP the single most critical component (+2.04 first-token, +1.00 argument loss), consistent with an early MLP that enriches token embeddings. In this model, then, attention mostly moves request information into the generating position in the first three layers, and MLPs at every depth transform it—the key–value memory picture of Geva et al. [41].

Arguments resolve late; unseen families never resolve. The logit lens [36] (Figure 2a) shows argument tokens on seen pages becoming decodable only from layer 7 on ($0.49 \rightarrow 0.90$ top-1 accuracy over layers 7–10), while the family token rises gradually (0.14 after layer 2, 0.67 at the output). On unseen pages the family token stalls at 0.17–0.22 from layer 5 onward: the later layers that sharpen seen-page predictions have no stored association to retrieve. Argument accuracy on unseen pages still reaches 0.48, matching the behavioural finding that the model copies entities from the request even when it picks the wrong tool.

Copy heads. Single-head ablations (Figure 3) are small individually (largest: L1H1 +0.19 first-token loss; L1H7 +0.035 argument loss), indicating redundancy. The copy score isolates a handful of early heads—L0H5 (0.14), L1H7 (0.11), L3H7 (0.10), L2H1 (0.09)—that attend

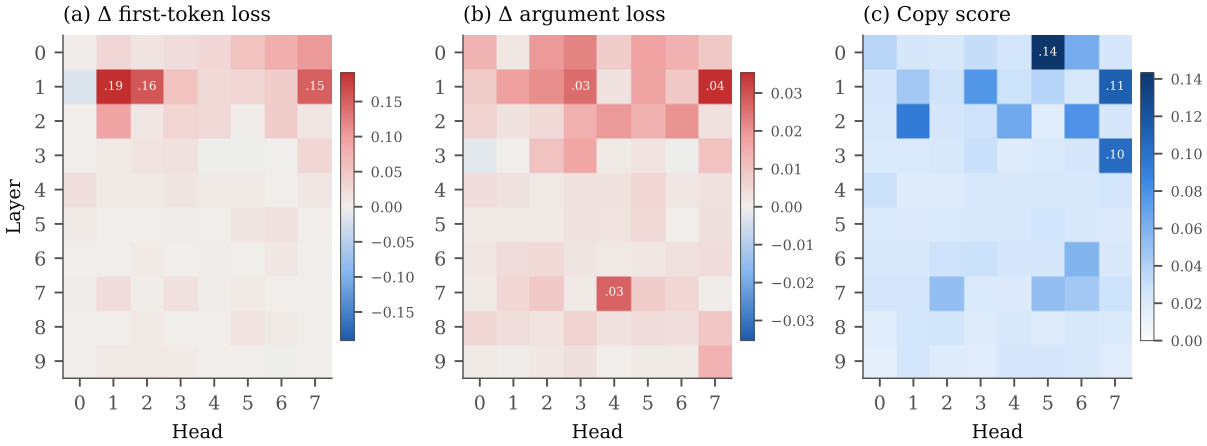


Figure 3: Per-head analysis on seen pages. **(a, b)** Loss change when a single head is zero-ablated. **(c)** Copy score: mean attention mass from a generating position to prompt tokens identical to the token being generated. The heads that matter, and the copying heads, are concentrated in layers 0–3.

		Seen loss		Unseen loss		`test_seen` exact	
Units	Rule	25%	50%	25%	50%	25%	50%
MLP neurons	random	1.342	3.155	4.128	4.833	0.140	0.000
	magnitude	1.351	3.415	4.120	4.944	0.175	0.000
	activation×weight	1.283	3.004	4.131	**4.691**	0.215	0.000
	Fisher (uniform)	1.274	**3.002**	4.138	4.736	0.205	0.005
	Fisher (global)	**1.257**	3.022	**4.112**	4.764	**0.220**	0.005
Attention heads	random	1.008	2.226	4.291	4.717	0.270	0.010
	magnitude	0.941	1.992	4.247	4.566	0.340	0.020
	activation×weight	0.933	1.901	4.252	4.537	0.360	0.055
	Fisher (uniform)	0.937	1.926	4.268	4.605	0.355	0.025
	Fisher (global)	**0.795**	**1.119**	**4.200**	**4.402**	**0.415**	**0.215**

Table 8: Fraction of units removed vs. quality, no recovery. Dense model: seen loss 0.705, unseen loss 4.146, `test_seen` exact 0.485 on this 200-request subset (Wilson interval ±0.07).

from the generating position to the matching prompt token, the copying behaviour described by Olsson et al. [40]. L1H7 is both a top copy head and the head whose removal most hurts argument loss.

10.3 How much can be removed

There are no free MLP neurons. Every scoring rule tracks random pruning closely (Figure 4a, Table 8): at 50% of MLP neurons removed, seen loss is 3.00–3.42 nats for all rules against 3.16 for random, and generation collapses to ≤ 0.005 exact for all. Even 10% removal costs 5–9 exact points (dense 0.485 $\rightarrow$ 0.400–0.440). The scores barely agree with each other: the bottom halves selected by Fisher and by random overlap at 0.507 (chance is 0.5), and only magnitude and activation, which share a weight-norm factor, overlap substantially (0.84). Fisher-global, free to choose per-layer budgets, spreads pruning almost uniformly (per-layer sparsity 0.45–0.53 at a 50% target). This is the opposite of the large-model regime, where 20–50% of structure can often be removed with little loss [28, 29, 33]: a 9.6M model trained on 100M tokens of its task has no spare MLP capacity, and every neuron carries a small share of the stored command knowledge.

Heads are prunable—if allocation is global. For attention heads, Fisher-global is in a different class (Figure 4b): removing 25% of heads (20 of 80; 0.66M parameters) costs 0.09 nats

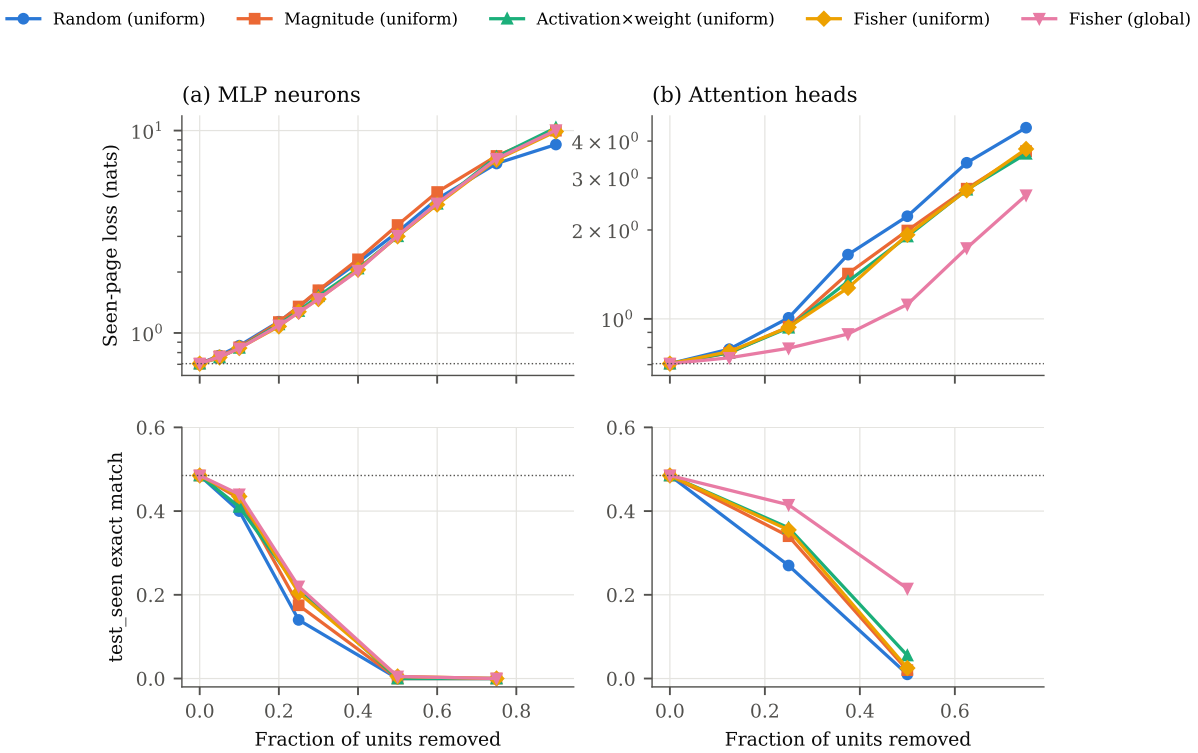


Figure 4: Post-training pruning without recovery. Top: seen-page loss (log scale); bottom: `test_seen` exact match (n=200). Dotted lines are the dense model. For MLP neurons **(a)**, no scoring rule meaningfully beats random; for attention heads **(b)**, the Fisher score with global allocation is far better than every alternative.

and 7 exact points, against 22 points for random; at 50%, it retains 0.215 exact where random retains 0.010. Its allocation is the mechanism: it removes *no* head from layers 0–2 and 62–88% of heads from layers 3–6 and 8–9 at the 50% target, mirroring the sublayer ablations. Uniform allocation, even with the same Fisher scores, is no better than magnitude, because it is forced to remove the critical early heads.

Pruning helps unseen-page loss slightly, and it is not knowledge. Removing up to 25% of MLP neurons, by any rule, leaves unseen-page loss flat or slightly *lower* (4.146 → 4.09–4.14) while seen-page loss nearly doubles. Unseen first-token loss falls from 5.90 to 5.67 under Fisher pruning at 30%. Generation on unseen pages, however, does not improve (command match 0.150 → 0.135–0.160 at 10%). We read this as a calibration effect: on unseen pages the dense model is confidently wrong, and damaging the network softens its predictions, which lowers cross-entropy without making them more correct.

10.4 Is page memory localized?

If the knowledge of seen pages lived in a small set of "page neurons" [42, 43], removing the units most important on seen pages but unimportant on unseen pages should hurt seen pages sharply and spare unseen pages. We rank units by the difference of their normalized Fisher scores on training requests and on `val_unseen` (scores and evaluation sets are disjoint) and compare against removing the most important seen-page units, the most *unseen*-specific units, and random units (Figure 5).

The prediction fails. At 20% removal, seen-specific pruning yields seen loss 1.153 against 1.126 for random; even removing the 20% of units with the *highest* seen-page Fisher score yields only 1.354, a +0.23-nat gap over random. Unseen loss stays within 0.10 nats of the dense model under all four rules, and at 10% the seen-specific model generates no better than random on

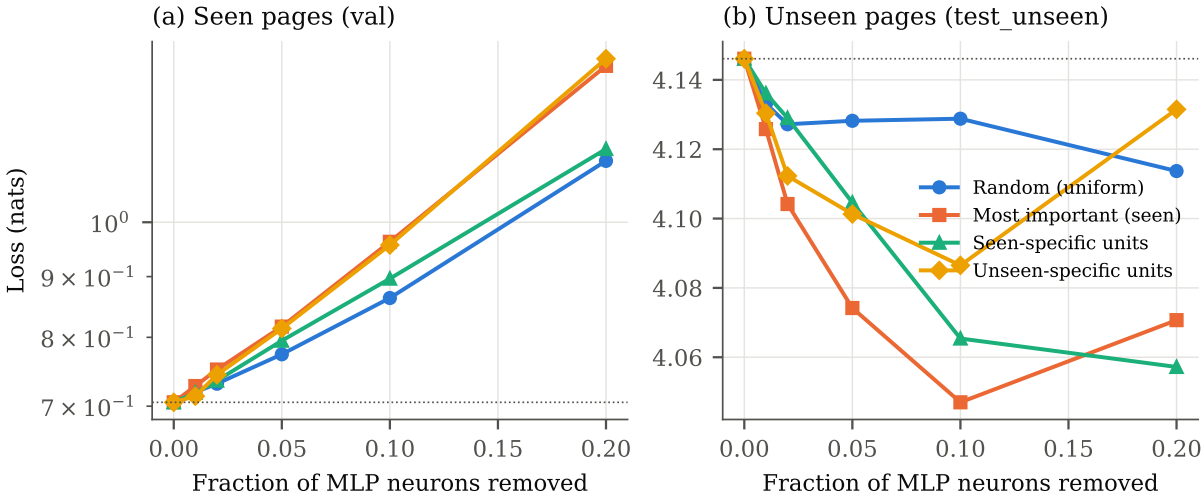


Figure 5: Differential pruning of MLP neurons. "Seen-specific" removes the units with the largest gap between normalized Fisher importance on seen and unseen pages; "most important (seen)" removes the top Fisher units.

MLP neurons removed	Params	Seen loss	Unseen loss	Exact	Command	Token F1	p50
none (dense)	9.57M	0.705	4.146	0.485	0.675	0.625	238 ms
50%, Fisher-global, no recovery	6.62M	3.022	4.764	0.005	0.125	0.071	–
50%, Fisher-global + 400 steps	6.62M	**1.565**	3.877	**0.160**	0.375	**0.408**	248 ms
50%, random + 400 steps	6.62M	1.636	3.898	0.150	**0.390**	0.374	238 ms
75%, Fisher-global, no recovery	5.15M	7.256	7.696	0.000	0.010	0.020	–
75%, Fisher-global + 400 steps	5.15M	2.227	**3.759**	0.025	0.265	0.275	199 ms

Table 9: Masked distillation after MLP pruning. Generation metrics on 200 `test_seen` requests. Parameters are those of the physically sliced model (max logit difference to the masked model $\leq 4 \cdot 10^{-6}$).

unseen pages (command match 0.150 vs. 0.160). Seen and unseen Fisher rankings correlate only weakly (Spearman 0.33). Within the resolution of this experiment, page knowledge in TinyShell is *distributed*: importance is spread thinly over most MLP units, matching recent doubts that memorization localizes to few units [45] and the observation that compression erodes long-tail knowledge first and broadly [35].

10.5 Recovery and real savings

Pruning without recovery is a probe, not a compression method. We therefore fine-tune pruned models with their masks fixed for 400 steps (batch 32, i.e. 12,800 training requests, about 1% of one epoch; AdamW, lr 10^{-4}) on $\frac{1}{2}$ cross-entropy plus $\frac{1}{2}$ KL divergence to the dense model's token distribution [31, 32], then *physically* slice the MLP matrices and time single-request decoding with the reference decoder (Table 9).

Three observations. First, recovery is real but partial at this budget: 50% pruning plus 400 steps returns from 0.005 to 0.160 exact (dense 0.485). Second, after recovery the scoring rule hardly matters (Fisher 0.160 vs. random 0.150, within noise at n=200), as one would expect if no subset of neurons is special. Third, *parameters are not latency*: the 6.62M sliced model decodes no faster than the dense one (248 vs. 238 ms p50) and the 5.15M model only 16% faster. At batch size 1 on a CPU, per-token cost is dominated by attention over the growing context, the 4,096 × 256 unembedding, and framework overhead, not by MLP arithmetic. Recovered models again show *lower* unseen-page loss than the dense model (3.76–3.90 vs. 4.15) with no gain in unseen-page generation, reinforcing the calibration reading of Section 10.3.

		Ten target families (n=200)			Other (n=183)
Model	Params	Exact	Command	Token F1	Exact
Dense generalist	9.57M	**0.465**	0.790	**0.625**	**0.470**
50% MLP, family scores, no recovery	6.62M	0.010	0.615	0.120	0.011
50% MLP, general scores, no recovery	6.62M	0.000	0.465	0.075	0.005
Specialist: 75% MLP, 25% heads + 400 steps	4.49M	0.395	0.895	0.605	0.000
Specialist: 90% MLP, 50% heads + 400 steps	2.95M	0.300	**0.910**	0.535	0.000

Table 10: Extracting a ten-family specialist from the 9.6M generalist. Parameters count the sliced MLPs and treat masked heads as removed. Without recovery, no mask preserves the target families; with 400 steps of family-data distillation, a 2.95M subnetwork keeps 65% of the dense model's exact match on its families and selects the family more often than the dense model. Wilson half-widths are about ±0.07.

10.6 Extracting a specialist

The negative results above concern a model asked to keep *all* of its 3,651 command families. A practical alternative is to keep only the families a given user needs. We score units by Fisher importance on 512 training requests from ten DevOps families (`git`, `docker`, `kubectl`, `systemctl`, `cargo`, `npm`, `pip`, `terraform`, `aws`, `helm`), remove MLP neurons and heads by global ranking, distill for 400 steps on the same family data, and evaluate on 200 held-out family requests (seen pages, held-out styles: `val`, `test_seen`, `test_paraphrase`) and on 183 `test_seen` requests from all other families (Table 10).

Before recovery, family-specific scores protect the target families slightly better than general scores (command match 0.615 vs. 0.465 at 50%), but no mask preserves exact match. After recovery, the specialists behave very differently from generally pruned models: the 4.49M specialist reaches 0.395 exact on its families (85% of the dense model's 0.465) and the 2.95M specialist 0.300 (65%), while both choose the right family *more* often than the dense model (0.895 and 0.910 vs. 0.790), because they no longer compete with thousands of other command families—the label-space effect of Section 7 reappearing. Everything else is forgotten (exact 0.000 on other families). By contrast, the 2.1M core model of Section 7, trained from scratch on ∼100 families, reached only 0.028–0.057 exact; the family sets and evaluation sets differ, so we read the comparison as directional evidence that *carving a specialist out of a larger generalist* beats training a specialist of similar size from scratch on this budget.

Finding (RQ5) Early attention (layers 0–2) reads the request and a few early heads copy its entities; MLPs at every depth hold command knowledge, and unseen-page predictions stall mid-network for lack of anything to retrieve. In this small-relative-to-data regime, MLP neurons are uniformly necessary (no scoring rule beats random; 10% removal costs 5–9 exact points) while attention heads are prunable when allocation is global (25% of heads for 7 points). Page knowledge is distributed rather than localized. Compression that keeps *everything* is therefore expensive here, but compression that keeps a *chosen subset* of families works: a distilled 2.95M subnetwork keeps 65% of the generalist's exact match on ten families.

11 Limitations and Threats to Validity

Synthetic evaluation. Requests are template paraphrases of a curated corpus (tldr common/linux only) with slot values from small pools; real-user performance is likely lower and slot robustness may partly reflect pool memorization. BM25's seen-split strength is an artifact of this design. **Metrics.** Exact match grants no semantic equivalence, `bash -n` checks syntax only, risk labels are heuristics, and nothing is executed. **Single seed.** Every training run uses seed 42; run-to-run variance is unmeasured, so small differences between runs (as opposed

to within-run paired comparisons, which our bootstrap covers) should be read as directional. **Confounded scaling.** Run 3 changes parameters, tokens, filters, and train styles at once; it is not a controlled ablation of run 2. **Core diagnostic.** The core run's first-token weighting tagged only the first active token per packed chunk (about one example in three), so its reported effect is a lower bound. **Pruning study.** Scores and masks are computed from 512 calibration examples and evaluated on 500 (loss) and 200 (generation) examples per split; generation metrics on 200 examples have Wilson half-widths up to ±7 points. All pruning results come from one checkpoint, recovery uses a small budget (400 steps), and latency is measured with the unbatched reference decoder. **Safety.** The model hallucinates flags, cannot verify local tool versions, and must not be given execution authority; the CLI never runs generated commands.

12 Future Work: Directions for Small Models

The pruning results suggest research directions that are specific to small models, where every parameter is in use.

- **Parser, not library.** If MLP capacity is spent storing a long tail of page knowledge that is distributed and cannot be pruned selectively, move that knowledge out of the weights. Train the same model with the retrieved tldr page of the target tool in context [16, 17] and test two predictions: unseen-page exact match rises, and the retrieval-conditioned model tolerates far more MLP pruning than Table 8 shows, because it no longer needs to store flags.
- **Generalist-to-specialist distribution.** Table 10 suggests shipping one generalist and carving per-user specialists on device from that user's command history. Open questions: how few calibration requests are needed, whether specialists can be merged, and how specialist quality scales with the number of families kept.
- **Non-uniform architectures from probes.** Heads matter in layers 0–2 and MLPs everywhere; at a fixed parameter count, a model with full attention early, few heads late, and the saved parameters moved into MLP width is a direct, cheap architecture test. Late attention layers are also candidates for weight sharing [58].
- **Train-time capacity reallocation.** Dynamic sparse training [59] could prune and regrow units during training, testing whether the "no free neurons" state is a property of the data or of dense training.
- **Calibration-aware routing.** Pruning lowers unseen-page loss by softening confidently wrong predictions; sequence log-probability or temperature-scaled confidence could route requests between model and retrieval, bounded above by the +7.1-point oracle gain of Section 9, and could trigger abstention on unseen tools.
- **Coverage-controlled data mix**: at fixed 100M tokens, trade variants per page for more pages, testing whether the unseen-loss regression (Table 5) is a mixing artifact.
- **Execution-grounded training** [18] and **semantic-equivalence evaluation** to target the 16.7% right-family-wrong-arguments rate.
- **Multiple seeds** for every headline comparison, and recovery budgets beyond 400 steps.

13 Released Platform

The repository contains the data pipeline (pinned corpus commit, manifests, data card), tokenizer and packing code with tested invariants, the 272-line model, training/evaluation/baseline code, per-run metadata, checkpoints for all four runs, raw per-example predictions, the significance script (`scripts/significance.py`), the probing and pruning harness (`scripts/prune_analysis.py`, `scripts/plot_pruning.py`), a packaged model, a CLI with retrieval mode and risk warnings, a training dashboard, and a 16-chapter learning book. Code is MIT-licensed; tldr data is CC-BY-4.0.

14 Conclusion

TinyShell shows that a 10M-parameter model trained in one CPU-day can become a usable suggestion engine *within* its training distribution, and that the same evaluation discipline that shows this also shows where it ends: page-level generalization. Opening the network sharpens that boundary. The model's MLPs are full of page knowledge, spread thinly enough that no neuron is free and no small set of neurons is the "memory" of a page; its attention is concentrated in a few early layers; and on unseen pages its computation simply has nothing to look up. Two design consequences follow for small language models. Capacity spent on storing a long tail of commands is capacity a model this size cannot spare, which argues for moving that knowledge out of the weights and into retrieved context; and a generalist is a good *source* of specialists, which argues for distributing one generalist and extracting per-user subnetworks from it. Both are now testable on the released platform.

Reproducibility

All numbers are generated by the repository's code and stored in `artifacts/`: evaluation `metrics.json`, `report.md`, and `significance.json`; training `metrics.jsonl`, `train.log`, `env.json`, `git_commit.txt`; and the pruning study's `pruning.json` and `scores.pt` under `artifacts/analysis/tinyshell-xl-10m/`. Chapter 15 of the learning book lists exact commands. The pruning study runs in about 2.4 hours on the same 4-vCPU host:

```
python scripts/prune_analysis.py
--run artifacts/runs/tinyshell-xl-10m
```

References

[1] Xi Victoria Lin, Chenglong Wang, Deric Pang, Kevin Vu, Luke Zettlemoyer, and Michael D. Ernst. Program synthesis from natural language using recurrent neural networks. Technical Report UW-CSE-17-03-01, University of Washington, Department of Computer Science and Engineering, 2017.

[2] Xi Victoria Lin, Chenglong Wang, Luke Zettlemoyer, and Michael D. Ernst. NL2Bash: A corpus and semantic parser for natural language interface to the Linux operating system. In *Proceedings of the Eleventh International Conference on Language Resources and Evaluation (LREC)*, 2018. arXiv:1802.08979.

[3] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

[4] Baptiste Rozière, Jonas Gehring, Fabrice Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Romain Sauvestre, Tom Raffel, et al. Code Llama: Open foundation models for code. *arXiv preprint arXiv:2308.12950*, 2023.

[5] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. LLaMA: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.

[6] Aaron Grattafiori et al. The Llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.

[7] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, 2022. arXiv:2203.02155.

[8] Ronen Eldan and Yuanzhi Li. TinyStories: Small language models that can tell stories. *arXiv preprint arXiv:2305.07759*, 2023.

[9] Peiyuan Zhang, Guangtao Zeng, Tiandi Li, Howard Wang, Lu Zhang, and Charles Zee. TinyLlama: An open-source small language model. *arXiv preprint arXiv:2401.02385*, 2024.

[10] Andrej Karpathy. nanoGPT: The simplest, fastest repository for training/finetuning medium-sized GPTs. https://github.com/karpathy/nanoGPT, 2023.

[11] Mario Michael Krell, Matej Kosec, Sergio P. Perez, and Andrew Fitzgibbon. Efficient sequence packing without cross-contamination: Accelerating large language models without impacting performance. *arXiv preprint arXiv:2107.02027*, 2021.

[12] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, 2022. arXiv:2205.14135.

[13] Jade Copet, Felix Kreuk, Itai Gat, Tal Remez, David Kant, Gabriel Synnaeve, Yossi Adi, and Alexandre Défossez. Simple and controllable music generation. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, 2023. arXiv:2306.05284.

[14] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020.

[15] Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, Tom Hinton, Eric Noland, Katie Millican, George van den Driessche, Bogdan Damoc, Aurelia Guy, Simon Osindero, Karen Simonyan, Jack W. Rae, Oriol Vinyals, Laurent Sifre, Seb Tighe, Sander Merity, Simon Hadfield, Charles Sumner, Sona Ortis-Baban, et al. Training compute-optimal large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, 2022. arXiv:2203.15556.

[16] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, 2020. arXiv:2005.11401.

[17] Sebastian Borgeaud, Arthur Mensch, Jordan Hoffmann, Trevor Cai, Eliza Rutherford, Katie Millican, George van den Driessche, Jean-Baptiste Lespiau, Bogdan Damoc, Aidan Clark, et al. Improving language models by retrieving from trillions of tokens. In *Proceedings of the 39th International Conference on Machine Learning (ICML)*, 2022. arXiv:2112.04426.

[18] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. InterCode: Standardizing and benchmarking interactive coding with execution feedback. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, 2023. arXiv:2306.14898.

[19] Yann LeCun, John S. Denker, and Sara A. Solla. Optimal brain damage. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 2, 1990.

[20] Song Han, Jeff Pool, John Tran, and William J. Dally. Learning both weights and connections for efficient neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 28, 2015. arXiv:1506.02626.

[21] Jonathan Frankle and Michael Carbin. The lottery ticket hypothesis: Finding sparse, trainable neural networks. In *International Conference on Learning Representations (ICLR)*, 2019. arXiv:1803.03635.

[22] Pavlo Molchanov, Stephen Tyree, Tero Karras, Timo Aila, and Jan Kautz. Pruning convolutional neural networks for resource efficient inference. In *International Conference on Learning Representations (ICLR)*, 2017. arXiv:1611.06440.

[23] Paul Michel, Omer Levy, and Graham Neubig. Are sixteen heads really better than one? In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 32, 2019. arXiv:1905.10650.

[24] Elena Voita, David Talbot, Fedor Moiseev, Rico Sennrich, and Ivan Titov. Analyzing multi-head self-attention: Specialized heads do the heavy lifting, the rest can be pruned. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 5797–5808, 2019. arXiv:1905.09418.

[25] Woosuk Kwon, Sehoon Kim, Michael W. Mahoney, Joseph Hassoun, Kurt Keutzer, and Amir Gholami. A fast post-training pruning framework for transformers. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, 2022. arXiv:2204.09656.

[26] Elias Frantar and Dan Alistarh. SparseGPT: Massive language models can be accurately pruned in one-shot. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, 2023. arXiv:2301.00774.

[27] Mingjie Sun, Zhuang Liu, Anna Bair, and J. Zico Kolter. A simple and effective pruning approach for large language models. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2306.11695.

[28] Xinyin Ma, Gongfan Fang, and Xinchao Wang. LLM-Pruner: On the structural pruning of large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, 2023. arXiv:2305.11627.

[29] Saleh Ashkboos, Maximilian L. Croci, Marcelo Gennari do Nascimento, Torsten Hoefler, and James Hensman. SliceGPT: Compress large language models by deleting rows and columns. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2401.15024.

[30] Mengzhou Xia, Tianyu Gao, Zhiyuan Zeng, and Danqi Chen. Sheared LLaMA: Accelerating language model pre-training via structured pruning. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2310.06694.

[31] Saurav Muralidharan, Sharath Turuvekere Sreenivas, Raviraj Joshi, Marcin Chochowski, Mostofa Patwary, Mohammad Shoeybi, Bryan Catanzaro, Jan Kautz, and Pavlo Molchanov. Compact language models via pruning and knowledge distillation. *arXiv preprint arXiv:2407.14679*, 2024.

[32] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*, 2015.

[33] Xin Men, Mingyu Xu, Qingyu Zhang, Bingning Wang, Hongyu Lin, Yaojie Lu, Xianpei Han, and Weipeng Chen. ShortGPT: Layers in large language models are more redundant than you expect. *arXiv preprint arXiv:2403.03853*, 2024.

[34] Andrey Gromov, Kushal Tirumala, Hassan Shapourian, Paolo Glorioso, and Daniel A. Roberts. The unreasonable ineffectiveness of the deeper layers. *arXiv preprint arXiv:2403.17887*, 2024.

[35] Sara Hooker, Aaron Courville, Gregory Clark, Yann Dauphin, and Andrea Frome. What do compressed deep neural networks forget? *arXiv preprint arXiv:1911.05248*, 2019.

[36] nostalgebraist. Interpreting GPT: The logit lens. LessWrong blog post, 2020.

[37] Nora Belrose, Zach Furman, Logan Smith, Danny Halawi, Igor Ostrovsky, Lev McKinney, Stella Biderman, and Jacob Steinhardt. Eliciting latent predictions from transformers with the tuned lens. *arXiv preprint arXiv:2303.08112*, 2023.

[38] Nelson Elhage, Neel Nanda, Catherine Olsson, Tom Henighan, Nicholas Joseph, Ben Mann, Amanda Askell, Yuntao Bai, Anna Chen, Tom Conerly, et al. A mathematical framework for transformer circuits. *Transformer Circuits Thread*, 2021.

[39] Kevin Wang, Alexandre Variengien, Arthur Conmy, Buck Shlegeris, and Jacob Steinhardt. Interpretability in the wild: a circuit for indirect object identification in GPT-2 small. In *International Conference on Learning Representations (ICLR)*, 2023. arXiv:2211.00593.

[40] Catherine Olsson, Nelson Elhage, Neel Nanda, Nicholas Joseph, Nova DasSarma, Tom Henighan, Ben Mann, Amanda Askell, Yuntao Bai, Anna Chen, et al. In-context learning and induction heads. *arXiv preprint arXiv:2209.11895*, 2022.

[41] Mor Geva, Roei Schuster, Jonathan Berant, and Omer Levy. Transformer feed-forward layers are key-value memories. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 5484–5495, 2021. arXiv:2012.14913.

[42] Damai Dai, Li Dong, Yaru Hao, Zhifang Sui, Baobao Chang, and Furu Wei. Knowledge neurons in pretrained transformers. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 8493–8502, 2022. arXiv:2104.08696.

[43] Kevin Meng, David Bau, Alex Andonian, and Yonatan Belinkov. Locating and editing factual associations in GPT. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, 2022. arXiv:2202.05262.

[44] Vitaly Feldman. Does learning require memorization? A short tale about a long tail. In *Proceedings of the 52nd Annual ACM SIGACT Symposium on Theory of Computing (STOC)*, pages 954–959, 2020. arXiv:1906.05271.

[45] Ting-Yun Chang, Jesse Thomason, and Robin Jia. Do localization methods actually localize memorized data in LLMs? A tale of two benchmarks. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*, 2024. arXiv:2311.09060.

[46] Niklas Stoehr, Mitchell Gordon, Chiyuan Zhang, and Owen Lewis. Localizing paragraph memorization in language models. *arXiv preprint arXiv:2403.19851*, 2024.

[47] tldr-pages contributors. tldr-pages: Simplified and community-driven man pages. https://github.com/tldr-pages/tldr, 2024. Licensed CC-BY-4.0; snapshot commit 95062b79e30fc3539246399b1dbfcfabbccc79cf.

[48] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in Neural Information Processing Systems (NeurIPS)*, 30, 2017. arXiv:1706.03762.

[49] Biao Zhang and Rico Sennrich. Root mean square layer normalization. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 32, 2019. arXiv:1910.07467.

[50] Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. RoFormer: Enhanced transformer with rotary position embedding. *arXiv preprint arXiv:2104.09864*, 2021.

[51] Noam Shazeer. GLU variants improve transformer. *arXiv preprint arXiv:2002.05202*, 2020.

[52] Ofir Press and Lior Wolf. Using the output embedding to improve language models. In *Proceedings of the 15th Conference of the European Chapter of the Association for Computational Linguistics (EACL)*, pages 157–163, 2017. arXiv:1608.05859.

[53] Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural machine translation of rare words with subword units. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 1715–1725, 2016. arXiv:1508.07909.

[54] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations (ICLR)*, 2019. arXiv:1711.05101.

[55] Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009.

[56] Edwin B. Wilson. Probable inference, the law of succession, and statistical inference. *Journal of the American Statistical Association*, 22(158):209–212, 1927.

[57] Bradley Efron and Robert J. Tibshirani. *An Introduction to the Bootstrap*. Chapman & Hall/CRC, 1993.

[58] Mostafa Dehghani, Stephan Gouws, Oriol Vinyals, Jakob Uszkoreit, and Łukasz Kaiser. Universal transformers. In *International Conference on Learning Representations (ICLR)*, 2019. arXiv:1807.03819.

[59] Utku Evci, Trevor Gale, Jacob Menick, Pablo Samuel Castro, and Erich Elsen. Rigging the lottery: Making all tickets winners. In *Proceedings of the 37th International Conference on Machine Learning (ICML)*, 2020. arXiv:1911.11134.

A Pruning Study Details

Scores. For MLP neuron i of layer ℓ with gate, up, and down weights $W^g, W^u \in \mathbb{R}^{768\times256}$, $W^d \in \mathbb{R}^{256\times768}$: magnitude $= \|W^g_{i,:}\| \, \|W^u_{i,:}\| \, \|W^d_{:,i}\|$; activation×weight $= \mathbb{E}_t|h_{t,i}| \, \|W^d_{:,i}\|$ over non-pad tokens t, where h is the SwiGLU hidden state; Fisher $= \mathbb{E}_x[(\partial\mathcal{L}(x)/\partial m_{\ell,i})^2]$ with $\mathcal{L}(x)$ the mean supervised-token loss of request x. Head scores are analogous, using the head's value and output-projection slices and the norm of its output vector. The differential score of a neuron is $\tilde{F}_u^{\text{unseen}} - \tilde{F}_u^{\text{seen}}$ with $\tilde{F} = F/\sum F$; the lowest values (most seen-specific) are removed first.

Copy score. For each supervised position t whose target token also occurs in the prompt, the attention mass head h places on the matching prompt positions, averaged over such positions.

Setting	Value
Checkpoint	run 3 `best` (step 3,500)
Units	7,680 MLP neurons (10 layers $\times$ 768), 80 heads (10 $\times$ 8)
Calibration, seen	512 random training requests, one per backward pass
Calibration, unseen	first 512 `val_unseen` requests
Calibration, families	512 training requests from the ten target families
Loss evaluation	first 500 `val` and `test_unseen` requests
Generation evaluation	first 200 `test_seen` and `test_unseen` requests, greedy
MLP sparsities	5, 10, 20, 25, 30, 40, 50, 60, 75, 90%
Head sparsities	12.5, 25, 37.5, 50, 62.5, 75%
Recovery	400 steps, batch 32, AdamW lr 10^{-4}, no decay, clip 1.0, loss $0.5\,\mathrm{CE} + 0.5\,\mathrm{KL}(p_{\mathrm{dense}} \,\|\, p_{\mathrm{pruned}})$, dropout 0.05
Decoding check	batched KV-cached vs. reference decoder: 100/100 identical
Slicing check	max $\lvert\Delta\mathrm{logit}\rvert$ masked vs. sliced $\leq 4 \cdot 10^{-6}$
Hardware / wall time	4 vCPU ARM64 (Neoverse-N1), 4 threads; 2.4 h end to end

Table 11: Configuration of the probing and pruning study (`scripts/prune_analysis.py`).